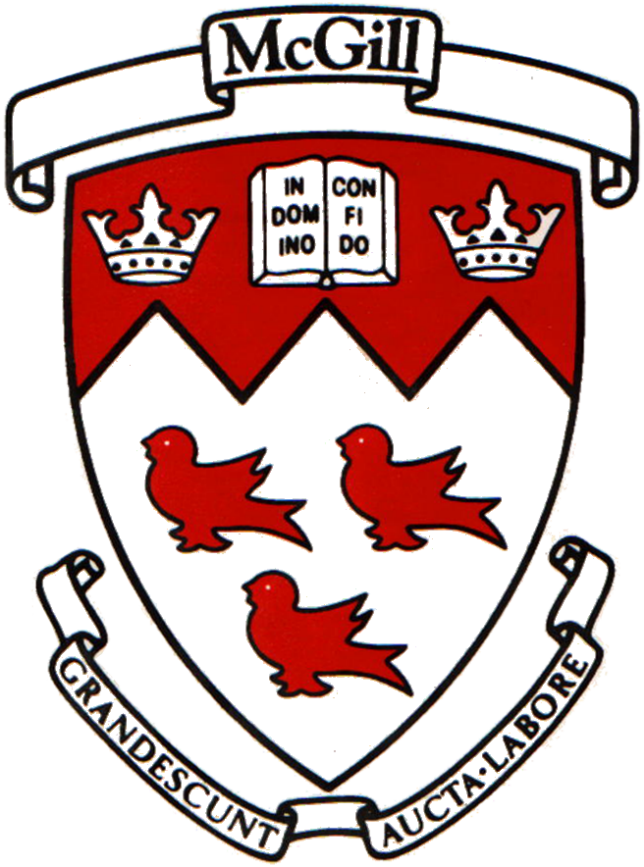
3D depth camera and robot navigation motion algorithm implementation

Jessica Xu, 261163415

Winter 2026



Project presented to Professor Joseph Vybihal

COMP 400

Contents

1 Introduction **3**

2 Background **3**

3 Getting the camera and lidar to work **4**

4 Algorithm ideas for the mapping and path planning of the robot **4**

4.1 Mapping (Probabilistic road map for path planning) 6

4.2 Rapidly exporing random tree (RRT) 7

5 The algorithms **8**

5.1 Model that can recognize a door 8

5.2 RRT 8

6 Testing **9**

6.1 The environment 9

6.2 Second algorithm (TO BE REMOVED) 9

1 Introduction

In robotics, the physical build is often viewed as the most complex procedure , but the true complexity lies in the digital brain that navigates the physical world. The ability to navigate an unmapped environment is the main core of autonomous robotics. In many real-world-scenarios, robots can't always rely on pre-loaded maps or GPS data. A couple examples include search-and-rescue mission and automated warehousing. These environments are often dynamic and unpredictable.

For the $R1D4$ project, our goal is to get the robot to the door in a randomly placed indoor classroom. Therefore, no initial map is given to the robot. To bridge the gap between raw environment and actionable data, and since we have access to both the Nuwa-HP60C 3D depth camera and a pulse Lidar, we have decided to add a dual-sensor system. The motivation behind the $R1D4$ project is to develop a robust simultaneous localization and mapping that does not move through space, but does it intelligently.

Since the first step of this project is to make the robot recognize the door, we will use the colored image feature on the 3D depth camera to feed the input of a trained model, then we will use the Lidar for the mapping algorithm as it is computationally lighter, more efficient and higher spacial accurate than the depth camera, so we can then use to map the path.

2 Background

To understand the navigation and recognition logic of the $R1D4$ project, it is necessary to establish some definitions. First, how does the Lidar work? The Lidar system operates on a Time-of-Flight (ToF) principle which consists of four cour subsystems: a laser emission system, a scanning system, a laser reception system and a signal processing system. Therefore, the laser will emit a laser beam towards a target, then the laser reception system will receive the signal back, which will be able to characterize the surface it hit, including the distance. The depth camera, just like its name, allows us to capture the depth in the picture, but also gives a colored image output.

The main challenge of this project is solving a SLAM problem. SLAM involves a robot building a map of an unknown environment while simultaneously keeping track of its own location.

Since $R1D4$ starts without a floor plan, it must use the sensors to create a set of data points in space to calculate its trajectory. Although no single sensor is perfect, Lidar is highly accurate for distance but blind to color and texture. This is where the camera comes into play, by providing a Coloured image and depth information, but can be sensitive to lighting and uses more processing power. By doing a sensor fusion, we can fuse these two sensors, with our approach of the camera identifying the target, while the Lidar provides obstacle avoidance data needed to reach it.

We will also train datasets and use YOLO for the recognition of the door in an image and use some very well known algorithm, such as the RRT algorithm, to get a valid path to the door. More details will be provided below.

3 Getting the camera and lidar to work

The model of the $3D$ Depth camera that we will use is a Nuwa-HP60C 3D depth camera. It uses a binocular structured light, which is designed for robotics and navigation. The model offers a $0.2m - 4m$ depth range, a $73.8°$ field of view and 640×480@30fps depth output. [cite: We learned that we can't be using ROS, therefore we will use an alternative way of operating the camera.

We had to find the SDK for both the Lidar and depth camera online, and it was a challenge to find the right one that works.

4 Algorithm ideas for the mapping and path planning of the robot

Before even starting to program, we had to establish some simple yet crucial assumptions. We first decided to assume that the surface the robot navigates one is a flat 2D surface. Then, the door remains open and is as narrow rectangle that sits between two parallel wall segment. Then, the obstacles should be at least the height of the camera/Lidar. The last assumption may be dropped throughout the object, which is whether obstacle will remain static or not.

Then, we needed to figure out how the robot can recognize where the door is in a indoor room. At first, we had the idea of not doing anything too complicated, that is create an algorithm that

recognizes the door by using step detection. We will take advantages of the gap between the left and right frame of the door. If there is a sudden increase in depth between them, then the door should be located there. Therefore, using the depth camera, we would be able to calculate the euclidean distance between two corners. Here is how it would look like pseudocode wise:

Algorithm to identify the door (step detection):
- Left frame (left side of the door): Sudden increase in depth.
- Gap: Empty space between the frames
- Right frame (right side of the door): Sudden decrease in depth.
- Calculate the euclidean distance between two corners (x_1, y_1) and (x_2, y_2). If the distance fits a doorframe distance, then most likely found a door.

This algorithm is simple, but also adds more assumptions on the classroom's structure, that is, gaps may exists between the walls, yet the door is not located there. It will most likely also make the overall path finding algorithm longer, since the robot has to go around the classroom. We therefore decided to use machine learning, which will train a model instead to recognize where the door is in a picture. We decided to take advantage of the colored feature on the 3D depth camera, and will most likely improve our experience.

Then, for the navigation algorithm, we had many plans. The simplest one being hugging the wall, also known as Frontier exploration. The robot will go around the classroom, while being at a certain distance from the wall, and once he sees the door, he will definitively being able to go through it since a door is mounted to a wall. Here is the idea:

- Spin 360° to get an initial 2D snapshot of the room. Find the nearest solid object. Therefore, scan the depth array, find the pixel with the smallest value, turn the robot until that distance is at your side, then sets up the wall hugging behavior. :
Greedy approach:
- Wall hugging: First, we need to maintain a distance from the nearest wall detected by the camera (let's choose $0.5m$).
- Constantly run the step detection algorithm above on every frame.
- If gets to a corner, and is path blocked, does another 360°, to continue the task above.
- Gap finding: Once detected the door
- Transition phase: Once gap is found, breaks the wall-hugging behavior. Gets to the midpoint

$M = (\frac{x_1+x_2}{2}, \frac{y_1+y_2}{2})$. Since want the robot to get out, will add a $1.5m$ extra behind M.
- Exit phase: Robot turns toward the center of the gap and drives through.
- If robot sees an obstacle, the robot will save the door's coordinate. Will hug the edge of the obstacle. After every few centimeters, check if the robot can see the midpoint M without any obstacle in the way. Once done, drive to M.

Now, this is definitively not an optimal algorithm, but idea wise, pretty simple to explain. This is a first look at the mapping of the robot. Then, for the path planning part, we were thinking of using the Dijkstra's algorithm. Dijkstra's is a greedy algorithm that allow us to find the shortest weighted path from a singular vertex to every other vertices it maps to.

Then, after upgrading the way the robot can see the door by using a trained model, we have decided to also upgrade the way the robot should map. We had to explore a couple algorithms that already exists, such as, Monte Carlo simulation, probability mapping, probability road map for path planning and Rapidly exploring random tree.

4.1 Mapping (Probabilistic road map for path planning)

Before starting with the algorithm, let us state some definitions. What is exactly Path Planning? Path planning is a computational problem that lets an autonomous mobile robot find the best or the most optimal path between two points. [cite: The goal is to find the shortest path between the two points, taking in consideration obstacles in its path.

The probabilistic Roadmap planning (PRM) is therefore used to determine the shortest path between two points, while avoiding collisions with obstacles. Since we will be using graphs, we will refer to these points as nodes (a.k.a. vertices). PRM consists of two phases: a construction phase and a query phase. In the construction phase, a network graph of possible paths based on free and occupied spaces is created. Here are the steps to create such network graph:

1. A random samples from the configuration space of the robot is created, therefore a random node is generated in the configuration space.

2. The system checks if the node lies in a free space or not. If the node is in a free space, it is will be added to the graph.

3. This new node will connect to the closest nodes through a straight line.

4. The system checks if the connection between the two nodes lies in a free space or not (if there is an obstacle or not). If it does, the connection is added to the graph between the two nodes.

Typically, it connects to k nearest neighbors. In the query phase, the start and goal configurations are connected to the graph. Therefore, at this stage, we were able to create a complete graph. To obtain the shortest path, we will then be using a Dijkstra's algorithm.

PRM is probabilistically complete, that is, the number of sampled nodes increases without bound, therefore the probability that the algorithm won't be able to find a path if there exists a path goes to 0.

We originally wanted to keep the maps in memory, but if the objects/obstacles move in the room, then the map in memory is considered useless, as the robot will need to redo the algorithm above. Therefore, we will scratch the idea of remembering the map.

4.2 Rapidly exporing random tree (RRT)

This is why instead of using PRM, we will use RRT. Since we are not in a static environment (the map does change everytime), it's more efficient to use RRT since it's best with single-query goals.

Now, the use of RRT is actually pretty straight forward [cite:]. We start by generating a set of random points, which will be connected to the closest vertices. That is, every time we create a new vertex, we need to check if the vertex is not part of the obstacle, including the edge that connects the two vertices or the neighborhood vertices. RRT allows us to create the mapping and find the shortest path.

RRT will produce graphs that are cubic. This is explained by the nodes that will be attached to their nearest neighbor. Therefore this will add some distances to the supposedly optimal distance, but the performance would be better than using rewirering the trees.

5 The algorithms

5.1 Model that can recognize a door

Instead of selecting a specific length for the door by using the depth array (as described in the idea above), it is better to train a model that can recognize what is a door in an image. First, we need to find a dataset to train the door detection model with. We first tried with a dataset composed of 400 inside door gpg images and labels, found on kaggle [3]. Since for this project, we are assuming the robot is indoor, we changed the dataset with a set of 581 images [insert source]. The labels are composed of a tuple of 5 values: the class, the localization of the point on an image, given by x, localization of the point on an image, given by y, width and height. We will use YOLO (you only look at it once) model instead of the conventional way of building a CNN, more precisely, we will fine-tune YOLO, that is, on top of an already trained model done by Ultralytics [2], we will add the specification to recognize what is an inside door.

First, we had to filter the dataset, since it was not solely composed of doors. The original dataset had 1213 images [1], but we had to remove two third of the dataset as it had other images that were not indoor doors. To avoid overfitting and increase the generalization, we had to use data augmentation on the pictures.

5.2 RRT

Our teammate already started working on a RRT algorithm which is based of a given map. He was able to successfully create a visual environment, given a map. The whole skeleton was already made, but a lot of changes had to be done, the most important one being that the robot moves every time it finds a new node, since no map will be given to the robot. We first had to change the data structure, then change how the robot moves, in every iteration. The implementation is now more accurate to a tree and cleaner, with a root, a tuple of (x, y) coordinates for each node, and a tuple of indices for the edges, connecting the parent and child nodes. We first did a running test on a simulated environment without incorporating the bias, and the number of iterations were much higher than if we included the bias. The bias is really useful when we know the position of the goal, in our case, the door. The results do change since nodes are randomly generated, but the bias was necessary and improved the number of steps it took for the robot to get to the door.

6 Testing

6.1 The environment

Since we can't work with the robot physically every time, we have decided to create a testing environment. The first simple testing environment consists of 4 things: the robot, the door, the empty space and the obstalces (including the walls). We created a $2D$ array of 0 (assuming everything is an empty space at first), then created the wall (denoted by 1's), then we randomly generated some obstacles which are also denoted by 1's. Finally, we randomly places the door (denoted by 2) and robot (denoted by 3), by assuming they are on the opposite sides of the array.

6.2 Second algorithm (TO BE REMOVED)

- Adds AI, probability rule, and Monte Carlo localization (MCL).

AI use:
- Identify the door by using a YOLO model, train the robot to recognize these.

The monte carlo localization is defined by: (add more details + clarification later on)
- The robot creates hundreds of "ghost version" of itself (particles) on its internal map.
- As the robot moves, compares to what it sees to what each ghost should see. If the particles don't match, match are killed off. Particles that match a survive.
- By doing this, the robot gains a confidence level in its position. The higher it is, then it can navigate to the door, even if it sees an obstacle.

Probability mapping:
- For every pixel in the map, give a value between 0 and 1 (like the probability space). Unknown probability is 0.5. Maintains a Bayesian probability map where each cell represents the likelihood of an obstacle. If the camera sees a chair, moves the probability from 0.5 to 0.7. The door is "the floor", with probability zero, walls have probabiltiy 1 (similar to ∞). This is why we want to minimize the cost.

Therefore the algorithm is as follows:
- Perceptions (CNN): Semantic labeling of objects (door vs obstacles).

- Estimation (MCL): Probabilistic localization using particle filters.

- Mapping: The robot identifies the closest unknown (0.5 probability), aka frontier exploration, and moves there to map it.

- MCL is ran constantly from the beginning, using the probabilistic map.

- While moving, uses the AI scans for the door label.

Global plannar use:

- On the probabilistic map, calculate the total cost $f(n) = g(n) + h(n)$, where $g(n)$ is the distance traveled (distance + risk) and $h(n)$ is the estimated distance to the door. Wants to choose the path with the lowest total cost.

- Safety factor: Add a risk cost to the map. Areas near chairs have a higher "cost", so the algorithm naturally picks a path that stays right in the middle of the wide-open space.

- Add a table of pros and cons.

References

[1] Miguel Arduengo, Carme Torras, and Luis Sentis. "Robust and adaptive door operation with a mobile robot". In: *Intelligent Service Robotics* (May 2021). ISSN: 1861-2784. DOI: 10.1007/s11370-021-00366-7. URL: http://dx.doi.org/10.1007/s11370-021-00366-7.

[2] Glenn Jocher, Jing Qiu, and Ayush Chaurasia. *Ultralytics YOLO*. Version 8.0.0. Jan. 2023. URL: https://github.com/ultralytics/ultralytics%7D.

[3] SayedMohamed. *Doors detection YOLO*. 2023. DOI: 10.34740/KAGGLE/DSV/6373320. URL: https://www.kaggle.com/dsv/6373320.